CS 3510 Section B: Design & Analysis of Algorithms August 21th 2024

Problem Set 1: Big-$\mathcal{O}$ and Divide & Conquer

Devin Fromond Due: August 30th 2024

- Please type your solutions using LaTeX or any other software. Handwritten solutions will not be accepted.
- Your algorithms must be in plain English & mathematical expressions, and the pseudo-code is optional. Pseudo-code, without sufficient explanation, will receive no credit.
- Unless otherwise stated, all logarithms are to base two.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.

1.) (20 points) For the following list of functions, cluster the functions of the same order (i.e., f and g are in the same group if and only if $f = \Theta(g)$) into one group, and then rank the groups in increasing order of growth. You can assume that the logs have a base of 2 unless specified otherwise. You do not have to justify your answer.

(a.) $n^{2\pi}$

(b.) $n^{\log\log n} + 3$

(c.) $n\log(2n) + 0.01n!$

(d.) $12n^{2^{2\log 10}} + n^2$ (Note: $2\log 10$ is an exponent to 2)

(e.) $2^{(\log(n))^2}$ (Note: $(\log(n))^2$ is an exponent to 2)

(f.) $1024^{3+\log n}$

(g.) $(log(n))^{log(n)}$

(h.) 2^{4096}

(i.) 2^{n+102}

(j.) $\log(n!)$

Solution: We will group functions of the same order, in increasing order of growth:

- 2^{4096}
- $(log(n))^{log(n)}, n^{log(log(n))} + 3$
- $log(n!)$
- $n^{2\pi}$
- $1024^{3+\log n}$
- $12n^{2^{2\log 10}} + n^2$
- $2^{log(n)^2}$
- 2^{n+102}
- $nlog(2n) + 0.01n!$

2.) (20 points) Suppose you are given the following inputs: a sorted array A, containing n distinct integers, a lower bound l, and an upper bound u, where l and u might not be in the array A.

Design a divide & conquer algorithm in $\mathcal{O}(\log n)$ to find the number of elements in A between l and u, inclusive. Make sure to justify the algorithm's correctness and the time complexity (using Master's Theorem).

Solution:

- *Algorithm:* Given the input of a sorted array, we will utilize properties of a binary search tree to complete the algorithm. The algorithm will perform two binary searches to find the index of the element at both the upper and lower bounds.

  First, we must perform a trivial check. If $u - l = 0$, the algorithm will return 0.

  The first binary search will find the index, call it l_i, of the element corresponding to the lower bound l. If l_i does not exist in the array, then the next element above l will be l_i.

  Similarly, the second binary search will find the index, call it u_i of the element corresponding to the upper bound u. If u_i does not exist in the array, then the next element below u will be u_i.

  Once the two indexes have been calculated, we have $Dist = u_i - l_i + 1$. *Dist* will be returned by the algorithm, assuming it has not returned 0 as a result of the trivial check.

  Recursively, the binary search algorithm has three components:
  - Base Case: If *len(A) == 1*, then we must return u_i or l_i, respectively, according to how the index bounds were defined previously.
  - Search Lower: If the comparison of $bound < A(\frac{n}{2})$ is true, then the algorithm will recursively call binary search on the first half of the array.
  - Search Upper: Else, then the algorithm will recursively call binary search on the second half of the array.
- *Correctness:* To prove correctness of the algorithm, we will use Proof by Induction.
  - Base Case: For an array of size *n==1*, the algorithm returns the element of the lower or upper bound, as defined by the aforementioned rules.
  - Inductive Hypothesis: We will assume the algorithm works correctly for any array of size n or smaller.
  - Inductive Step: For any size of *n+1*, the algorithm recursively calls binary search on half of the array to identify the upper and lower bound indexes.

- *Time Complexity Analysis:* In our recursive binary search method, we are doing a constant time of work at each step: that is, checking the *len(A)==1* equality and assigning the indexes, if necessary, is O(1). At each level, the number of elements is $\frac{n}{2}$, where n is the elements in the previous level. As such, we have: $T(n) = T(\frac{n}{2}) + O(1)$. Generally, we have the form as defined by the Master's Theorem as $T(n) = aT(\frac{n}{b}) + O(n^d)$. Therefore, we find *b=2*, *a=1*, and *d=0*. By the Master's Theorem, since $log_b a = log_2 1 = 0 = d$, the algorithm is $O(n^d log(n)) = O(n^0 log(n) = O(log(n))$ time complexity.

3.) (20 points) Assume n is a power of 4. Assume we are given an algorithm $f(n)$ as follows:

```
function f(n):
   if n>1:
      for i in range(20):
        f(n/4)
      for i in range(n*n):
        print("Banana")
      f(n/4)
      f(n/4)
   else:
      print("Monkey")
```

(a.) What is the running time for this function $f(n)$? Justify your answer. (Hint: Recurrences)

Solution: $T(n) = 20T(\frac{n}{4}) + O(n^2) + T(\frac{n}{4}) + T(\frac{n}{4})$

The provided function contains a for loop that recursively calls f(n/4) 20 times, resulting in $20T(\frac{n}{4})$. Then, there is a for loop that runs n*n times, resulting in $O(n^2)$. Then, there are two recursive calls to f(n/4) resulting in two $T(\frac{n}{4})$ terms.

Simplifying the terms, we have $T(n) = 22T(\frac{n}{4}) + O(n^2)$.

Generally, we have the form as defined by the Master's Theorem as $T(n) = aT(\frac{n}{b}) + O(n^d)$. Therefore, we find *b=4*, *a=22*, and *d=2*. By the Master's Theorem, since $log_b a = log_4 22 \approx 2.23 > 2$, the algorithm is $O(n^{log_b(a)}) = O(n^{log_4(22)}) \approx O(n^{2.23})$ time complexity.

(b.) How many times will this function print "Monkey"? Please provide the exact number in terms of the input n. Justify your answer.

Solution: Each recursive call of $f(n/4)$ will eventually satisfy the condition n ≤ 1 since n is repeatedly divided by 4..

We can analyze the branching of the function at different values of n:

- For n = 1: *Monkey* will print 1 time.
- For n = 4: *Monkey* will print 22 times.
- For n = 16: *Monkey* will print 484 times.
- For n = 64: *Monkey* will print 10648 times.

The number of times *Monkey* is printed follows the pattern 22^y where y is related to $log_4(n)$. Looking at the algorithm, each distinct recursive call will eventually reach the base case and print *Monkey*. Since there are 22 recursive at each level in the function *f(n)*, the total number of times *Monkey* is printed is determined by the total number of calls at the deepest level of recursion, which is $log_4(n)$ levels deep.

Therefore, we find *Monkey* is printed $22^{log_4(n)}$ times.

4.) (20 points) Kartik and Richard are trying to come up with Divide & Conquer approaches to a problem with input size n. Kartik comes up with a solution that utilizes 32 subproblems, each of size $n/16$ with time $n\sqrt[4]{n} + 2n \log n$ to combine the subproblems. Meanwhile, Richard comes up with a solution that utilizes 24 subproblems, each of size $n/5$ with time $3n^2 + n \log^2 n$ to combine.

(a.) What is the runtime of both algorithms? Which one runs faster, if either?

Solution: We will begin with Kartik. Utilizing the Master's Theorem, we know $a =$ *number of recursive problems* and $b =$ *size of subproblem.* Therefore, we have $a = 32$ and $b = 16$. Inputting into the Master's Theorem yields $T(n) = 32T(\frac{n}{16}) + O(n^{5/4} + 2n)$. We find $d = \frac{5}{4}$. Therefore, since $log_{16}(32) = 1.25 = d$, Kartik's algorithm has a runtime of $O(n^{\frac{5}{4}} log(n))$.

Now, we will solve for Richard. Utilizing the Master's Theorem, we have $a = 24$ and $b = 5$. Inputting into the Master's Theorem yields $T(n) = 24T(\frac{n}{5}) + O(3n^2 + nlog^2 n)$. We find $d = 2$. Therefore, since $log_5(24) \approx 1.97 < d = 2$, Richard's algorithm has a runtime of $O(n^2)$.

Therefore, we can see Kartik's algorithm is faster.

(b.) Let's say Shresha also tries to solve the same problem using an algorithm of her own. She utilizes 2 sub-problem of size $\sqrt{n}$ with constant time to combine the subproblems. She defines a base case where $n = 2$. What is the runtime of this algorithm? Is this algorithm faster than the one you chose in part (a)? (Hint: you cannot simply plug in the Master Theorem. Think about how the Master Theorem is derived for this question.)

Solution: We will first define the recurrence relation for Shresha's algorithm as $T(n) = 2T(\sqrt{n}) + O(1)$. We will simplify this relation by letting $m = \sqrt{n}$. Thus, we have $T(m^2) = 2T(m) + O(1)$.

To determine the work done at each level, we have $T(\sqrt{n}) = 2T(n^{1/4}) + O(1)$, then we have $T(n) = 2T(n^{1/4}) + 2O(2) + O(1)$. This trend continues as we solve for $T(n^{1/4})$ and more.

Therefore, size of the work at a level k is $n^{1/2^k}$. To reach the base case, we have $n^{1/2^k} = 2$. Taking the log on both sides, we have $\frac{log(n)}{2^k} = log(2)$. Therefore, we have $k = log(log(n))$. Each level of recursion tree does constant work, but there are 2^k nodes at each level k. Therefore, the total work done at each level is $O(k^2)$.

The total work done at all levels is as follows:

$$TotalWork = O(2^0) + O(2^1) + O(2^2) + ... + O(2^k)$$

This is a geometric series such that $k = log(log(n))$ and so the total work is bounded by $O(2^k) = O(log(n))$.

Therefore, the runtime of Shresha's algorithm is $O(log(n))$, which is faster than part A.

5.) (20 points) Assume that n is a power of two. The Hadamard matrix H_n is defined as follows:

$$H_1 = \begin{bmatrix} 1 \end{bmatrix}$$
$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$
$$H_n = \begin{bmatrix} H_{n/2} & H_{n/2} \\ H_{n/2} & -H_{n/2} \end{bmatrix}$$

Design an $O(n \log n)$ algorithm that calculates the vector $H_n v$, where n is a power of 2 and v is a vector of length n. Justify your algorithm's correctness and its runtime by providing a recurrence relation and solving it. (Hint: you may assume adding two vectors of order n takes $\mathcal{O}(n)$ time.)

Solution:

- *Algorithm:* We will begin by dividing the work into two. To start, we split the vector v into two halves: v_1 and v_2, each having a length of $\frac{n}{2}$.

  Next, we will compute two Hadamard products as vectors: u_1 and u_2. These vectors are defined as $u_1 = H_{n/2} \cdot (v_1 + v_2)$ and $u_2 = H_{n/2} \cdot (v_1 - v_2)$. Therefore, combining u_1 and u_2 will yield $H_n \cdot v = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$.

  This algorithm will have a base case of n==1 which will return v, the vector. The vector matrix will then be partitioned and two recursive calls, each having half of the Hadamard matrix as one parameter and the other parameter of the matrix addition/-subtraction as previously defined.

- *Correctness:* To prove correctness of the algorithm, we will use Proof by Induction.
  - Base Case: For a matrix of side $n = 1$, the algorithm correctly returns the vector v. Since the Hadamard matrix is just [1], multiplying by v returns v.
  - Inductive Hypothesis: We will assume the algorithm works correctly for any matrix of size $n = 2k$ or smaller, where k is an integer.
  - Inductive Step: For a matrix of size $n = 2k$, where n is a power of 2, the algorithm breaks down the matrix into two sub-matrices, each of size k. The matrix H_n has a structure of being composed of four smaller H_k matrices.

- *Time Complexity Analysis:* The recurrence relation follows $T(n) = 2T(\frac{n}{2}) + O(n)$ since the work done at each level is $O(n)$ (as a result of the vector addition/subtraction) and there are 2 recursive calls for each level. By the Master's Theorem, we have $a = 2$, $b = 2$, and $d = 1$. As such, the time complexity for the algorithm is $O(nlog(n))$ since $log_2(2) = 1 = d$.